

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Dipartimento di Informatica – Scienza e Ingegneria

Corso di Laurea Magistrale in
Informatica

PoS tagging italiano con BERT

Progetto di Natural Language Processing

Studente: Pierre Errico

Matricola: 0001235580

Anno Accademico 2026/2027

Abstract

Il progetto affronta il fine-tuning di BERT italiano per il PoS tagging sul benchmark UD Italian-ISDT 2.15, con l'obiettivo di valutarne le prestazioni rispetto a UDPipe 2 e ai risultati della letteratura. Il compito principale consiste nella predizione dei tag UPOS a partire da parole e confini di frase gold, mantenendo gli split ufficiali del corpus e confrontando i sistemi mediante accuracy, macro-F1 e analisi degli errori. Per approfondire il contributo dell'adattamento del modello e della decodifica strutturata, gli esperimenti comprendono un encoder congelato, BERT fine-tuned e una variante con CRF, valutati su tre seed sia sul solo UPOS sia dopo l'estensione alla predizione delle caratteristiche morfologiche `FEATS`.

BERT fine-tuned raggiunge prestazioni UPOS in linea con i risultati della letteratura e valori di accuracy e macro-F1 vicini a quelli di UDPipe 2. L'estensione morfologica si accompagna a un miglioramento delle prestazioni UPOS e consente di ottenere risultati confrontabili con il riferimento anche nella predizione delle caratteristiche `FEATS`. I confronti interni mostrano il contributo del fine-tuning rispetto all'encoder congelato e un beneficio più circoscritto del CRF sulla macro-F1 nella prima fase. L'analisi degli errori evidenzia comunque difficoltà sulle caratteristiche poco frequenti e permette di affiancare alla correttezza complessiva una lettura delle prestazioni sulle singole classi.

Indice

1 Introduzione **1**
1.1 Annotazione di categorie grammaticali . . . 1
1.2 Le principali sfide del PoS tagging . . . 3
1.3 Modelli neurali per il linguaggio . . . 3
1.4 Architettura BERT . . . 4
1.4.1 Tokenizzazione . . . 4
1.4.2 Struttura dell'encoder . . . 4
1.4.3 Preaddestramento . . . 6
1.5 Obiettivi del progetto . . . 6

2 Metodologia **7**
2.1 Definizione dell'esperimento . . . 7
2.2 Dati e annotazioni . . . 8
2.2.1 Corpus e suddivisione . . . 8
2.2.2 Rappresentazione di UPOS e FEATS . . . 9
2.3 Preprocessing e allineamento . . . 9
2.4 Modelli e funzione obiettivo . . . 9
2.5 Procedura di addestramento . . . 10
2.6 Selezione del checkpoint . . . 10
2.7 Valutazione . . . 11
2.7.1 Metriche UPOS . . . 12
2.7.2 Metriche FEATS . . . 12
2.7.3 Aggregazione e analisi degli errori . . . 13
2.8 Confronto con UDPipe 2 . . . 13
2.9 Riproducibilità . . . 13

3 Risultati **14**
3.1 Confronto UPOS con UDPipe 2 . . . 14
3.2 Posizionamento rispetto alla letteratura . . . 16
3.3 Confronti interni tra le configurazioni BERT . . . 17
3.4 Estensione alla predizione `FEATS` . . . 18
3.5 Analisi degli errori . . . 19
3.5.1 Esempi contestuali . . . 20

4 Conclusioni **21**

Bibliografia **22**

Capitolo 1

Introduzione

1.1 Annotazione di categorie grammaticali

Nella linguistica computazionale, il Part-of-Speech (PoS) tagging è un compito di etichettatura di sequenze di token, nel quale a ogni token viene assegnata un'etichetta che ne identifica la categoria grammaticale nel contesto della frase. Un token corrisponde generalmente a una parola; può tuttavia indicare anche altri elementi linguistici, come simboli e punteggiatura. Il PoS tagging riveste un ruolo fondamentale per l'analisi linguistica e costituisce una fase preliminare per numerose applicazioni di Natural Language Processing (NLP), tra cui la lemmatizzazione, l'analisi sintattica e la traduzione automatica [1].

Universal Dependencies (UD) è un framework per l'annotazione linguistica che definisce convenzioni condivise per la rappresentazione delle categorie grammaticali, delle caratteristiche morfologiche e delle relazioni sintattiche in numerose lingue, tra cui l'italiano [2]. La standardizzazione delle annotazioni e la disponibilità di corpora multilingue ne favoriscono l'impiego nella ricerca e nello sviluppo di modelli NLP.

In particolare, UD definisce un inventario di 17 categorie grammaticali universali, denominate Universal Part-of-Speech (UPOS) tags [3]. Le caratteristiche morfologiche, come genere, numero e tempo verbale, sono rappresentate separatamente, quando applicabili, mediante gli attributi del campo `FEATS` [4]. La Tabella 1.1 riporta le categorie UPOS, accompagnate da esempi contestuali in italiano e dagli attributi FEATS attestati nel corpus annotato UD Italian-ISDT (Italian Stanford Dependency Treebank) v2.15. Questi ultimi comprendono tutti gli attributi presenti almeno una volta nelle annotazioni di ciascuna categoria, anche in casi rari, e non sono pertanto necessariamente applicabili a tutti i token appartenenti alla stessa categoria.

Tabella 1.1: Categorie UPOS, esempi contestuali in italiano e attributi morfologici FEATS attestati nel treebank UD Italian-ISDT v2.15. Il trattino indica l'assenza di attributi attestati o di un esempio contestuale. Per `PART` non è riportato un esempio in italiano, poiché nel corpus il tag è utilizzato per il marcatore possessivo inglese.

Tag UPOS	**Categoria grammaticale**	**Esempio contestuale**	**FEATS attestati**
`ADJ`	Aggettivo	Il gatto è **nero**.	`Degree`, `Gender`, `Number`, `NumType`, `Poss`, `PronType`, `Typo`
`ADP`	Adposizione	Vado **a** casa.	`Gender`, `Number`
`ADV`	Avverbio	Il gatto dorme **tranquillamente**.	`Degree`, `Gender`, `Number`, `PronType`
`AUX`	Verbo ausiliare	Maria **ha** mangiato.	`Gender`, `Mood`, `Number`, `Person`, `Tense`, `VerbForm`
`CCONJ`	Congiunzione coordinante	Leggo **e** scrivo.	–
`DET`	Determinante	**Il** gatto dorme.	`Definite`, `Gender`, `Number`, `Poss`, `PronType`
`INTJ`	Interiezione	**Oh**, sei arrivato!	`Polarity`
`NOUN`	Nome comune	Il **gatto** dorme.	`Foreign`, `Gender`, `Number`, `Tense`, `VerbForm`
`NUM`	Numerale	Ho **tre** libri.	`NumType`
`PART`	Particella	–	–
`PRON`	Pronome	**Lei** legge un libro.	`Clitic`, `Gender`, `Number`, `Person`, `Poss`, `PronType`
`PROPN`	Nome proprio	**Maria** legge un libro.	`Foreign`, `Gender`, `Number`, `NumType`
`PUNCT`	Punteggiatura	Sei a casa**?**	–
`SCONJ`	Congiunzione subordinante	Resto a casa **perché** piove.	`PronType`
`SYM`	Simbolo	Il risultato è 2 + 3.	–
`VERB`	Verbo	Il gatto **dorme**.	`Gender`, `Mood`, `Number`, `Person`, `Tense`, `VerbForm`
`X`	Altro	Ha pronunciato **xfgh**.	`Foreign`, `Gender`, `Number`

A titolo di esempio, si consideri la frase «*Il gatto dorme.*». Nell'ambito del PoS tagging, ai quattro token viene assegnata rispettivamente la sequenza di etichette

`DET NOUN VERB PUNCT`. A queste annotazioni si aggiungono, quando applicabili, le informazioni morfologiche rappresentate nel campo `FEATS`. Il token «*gatto*», ad esempio, è classificato come `NOUN` e presenta gli attributi `Gender=Masc|Number=Sing`, che ne specificano il genere maschile e il numero singolare.

1.2 Le principali sfide del PoS tagging

Una delle principali difficoltà del PoS tagging è rappresentata dall'ambiguità grammaticale: una stessa parola può appartenere a più categorie, e la scelta del tag corretto dipende dal contesto [1]. Ad esempio, la parola *porta* può essere un nome («*La porta è chiusa.*») o un verbo («*Lui porta il libro.*»).

Un altro problema risiede nello sbilanciamento della distribuzione dei tag: alcune categorie, come nomi comuni (`NOUN`), determinanti (`DET`) e adposizioni (`ADP`), sono molto frequenti, mentre altre, come simboli (`SYM`) ed elementi non classificabili nelle altre categorie (`X`), compaiono raramente [5]. La presenza di classi con pochi esempi può rendere più difficile l'apprendimento di modelli robusti e generalizzabili [6].

A ciò si aggiungono ulteriori problematiche:

- la presenza di parole rare o non osservate durante l'addestramento, tra cui neologismi, termini specialistici e nomi propri, che possono risultare difficili da classificare. Alcune di queste possono inoltre essere assenti dal vocabolario del modello (Out-of-Vocabulary, OOV) [7];
- le differenze tra il dominio dei testi di addestramento e quello dei testi da analizzare, che possono ridurre la capacità di generalizzazione del modello [1];
- la predizione degli attributi morfologici del campo `FEATS`, che introduce ulteriori difficoltà dovute alla presenza di attributi applicabili soltanto a determinate categorie grammaticali e alla distribuzione sbilanciata dei relativi valori;
- gli eventuali errori o incoerenze nelle annotazioni, che possono influenzare sia l'apprendimento sia la valutazione [1].

Queste problematiche motivano l'impiego di modelli capaci di costruire rappresentazioni linguistiche sensibili al contesto e di sfruttare conoscenze acquisite attraverso il preaddestramento su grandi quantità di dati testuali [8].

1.3 Modelli neurali per il linguaggio

L'evoluzione delle tecniche di NLP ha progressivamente migliorato la modellazione del contesto, passando da rappresentazioni lessicali statiche ad architetture in grado di integrare informazioni contestuali sempre più complesse [9].

Tra gli approcci neurali più influenti, Word2Vec [10] ha permesso di apprendere rappresentazioni vettoriali dense delle parole (i.e. *word embeddings*), nelle quali termini utilizzati in contesti simili tendono ad assumere rappresentazioni vicine.

Tali embedding sono tuttavia statici, ossia associano a ciascuna parola un unico vettore indipendentemente dal contesto, senza distinguere i diversi significati o ruoli grammaticali che essa può assumere.

Le reti neurali ricorrenti (RNN) producono rappresentazioni dipendenti dal contesto attraverso l'elaborazione sequenziale dei token. Le Long Short-Term Memory (LSTM) migliorano la capacità di modellare dipendenze a lungo termine, mentre le varianti bidirezionali, come le BiLSTM, integrano informazioni provenienti da entrambe le direzioni della sequenza [9, 7]. La natura ricorrente di queste architetture limita tuttavia la parallelizzazione e può rendere difficile la modellazione di dipendenze particolarmente distanti [11].

L'architettura Transformer [11] affronta alcuni di questi limiti sostituendo la ricorrenza con meccanismi di attenzione, tra cui la *self-attention*, che permette di modellare direttamente le relazioni tra i token, anche distanti tra loro, favorendo un'elaborazione parallela della sequenza e la produzione di rappresentazioni contestuali.

1.4 Architettura BERT

Tra i modelli basati sull'architettura Transformer, Bidirectional Encoder Representations from Transformers (BERT) [8] sfrutta un Transformer Encoder-only bidirezionale per produrre rappresentazioni contestuali dei token, risultando particolarmente adatto a compiti come il PoS tagging, nei quali la classificazione grammaticale dipende dal contesto frasale.

1.4.1 Tokenizzazione

L'input dell'encoder di BERT è costituito da token prodotti da WordPiece, un metodo di tokenizzazione che mantiene le parole intere oppure le suddivide in frammenti più piccoli, detti *subword*, selezionati da un vocabolario predefinito. Ad esempio, il tokenizer associato al modello `dbmdz/bert-base-italian-cased` rappresenta *bicicletta* con un singolo token, mentre suddivide *casetta* in `case` e `##tta` (il prefisso `##` indica la continuazione della parola). Questo approccio permette di rappresentare parole rare combinando frammenti già disponibili [8].

1.4.2 Struttura dell'encoder

Il modello `dbmdz/bert-base-italian-cased` adotta la configurazione BERT-Base, composta da 12 blocchi Transformer Encoder, 12 teste di attenzione per blocco e rappresentazioni vettoriali di dimensione 768 [12]. Le rappresentazioni vettoriali dei token (i.e. *token embeddings*) vengono sommate agli embedding posizionali e di segmento, che codificano rispettivamente la posizione nella sequenza e l'appartenenza ai segmenti di testo. Le rappresentazioni risultanti attraversano i 12 blocchi in successione [8]. La Figura 1.1 riassume il flusso delle rappresentazioni e la struttura interna di ciascun blocco.

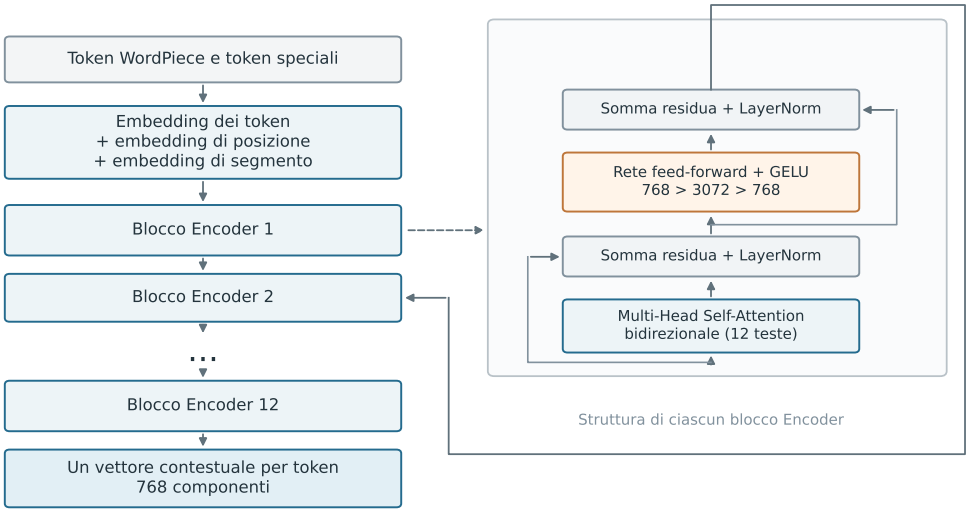


Figura 1.1: Struttura dell'encoder BERT-Base utilizzato nel progetto, con 12 teste di attenzione per blocco e vettori di 768 componenti. A sinistra, il flusso delle rappresentazioni procede dall'alto verso il basso attraverso i 12 blocchi; a destra, la struttura interna comune a tutti i blocchi, con attenzione bidirezionale, rete feed-forward e connessioni residue. La freccia di uscita mostra, come esempio, il passaggio dal primo al secondo blocco. Schema basato sull'architettura descritta da Devlin et al. [8].

Ogni blocco comprende una Multi-Head Self-Attention e una rete feed-forward. La prima esegue più operazioni di attenzione in parallelo, mediante proiezioni query, key e value apprese per ciascuna testa. Le uscite vengono concatenate e proiettate nello spazio di dimensione 768 [11]. Per ciascuna testa, l'attenzione è calcolata come:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d_k}}\right)V, \tag{1.1}$$

dove Q, K e V sono matrici ottenute mediante proiezioni lineari apprese delle rappresentazioni in ingresso:

- **Query (Q):** rappresentano le informazioni ricercate dai token;
- **Key (K):** rappresentano le caratteristiche utilizzate per calcolare la compatibilità con le query;
- **Value (V):** contengono le informazioni da aggregare mediante i pesi di attenzione.

Il prodotto QK^{T} determina i punteggi di compatibilità, scalati per $\sqrt{d_k}$, dove $d_k = 768/12 = 64$ è la dimensionalità delle key per ciascuna testa. La softmax normalizza tali punteggi in pesi di attenzione, utilizzati per calcolare una combinazione pesata dei vettori value.

La rete feed-forward applica indipendentemente a ciascun token due trasformazioni lineari, da 768 a 3 072 componenti e viceversa, separate da un'attivazione GELU.

Entrambi i sottolivelli impiegano dropout, connessioni residue e Layer Normalization per favorire la regolarizzazione e la stabilità dell'addestramento [8, 11]. Al termine dei 12 blocchi, l'encoder restituisce un vettore contestuale di 768 componenti per ciascun token, utilizzabile nei compiti supervisionati.

1.4.3 Preaddestramento

Nel BERT originale il preaddestramento combina il Masked Language Modeling (MLM) e la Next Sentence Prediction (NSP). Nel MLM viene selezionato casualmente il 15% dei token della sequenza: di questi, l'80% viene sostituito con il token speciale `[MASK]`, il 10% con un token casuale e il restante 10% rimane invariato. Il modello deve predire i token originali nelle posizioni selezionate, sfruttando il contesto bidirezionale. La NSP richiede invece di determinare se due segmenti di testo siano consecutivi nel corpus, mediante una classificazione binaria [8].

Attraverso il preaddestramento, BERT apprende rappresentazioni linguistiche contestuali da grandi quantità di testo, che possono successivamente essere adattate a diversi compiti supervisionati mediante fine-tuning. In particolare, il checkpoint adottato è stato preaddestrato su corpora italiani e mantiene la distinzione tra lettere maiuscole e minuscole durante la tokenizzazione [12].

1.5 Obiettivi del progetto

Il lavoro adatta e valuta BERT preaddestrato per il PoS tagging italiano, confrontandolo con sistemi di riferimento sul benchmark UD Italian-ISDT 2.15. Il compito principale consiste nella predizione dei tag UPOS, con gli split ufficiali, tokenizzazione e confini di frase gold. Il confronto diretto con UDPipe 2 sul medesimo test è affiancato dall'analisi dei risultati riportati in letteratura, considerando le differenze di release, condizioni di input e metriche che ne limitano la comparabilità [5, 13].

Gli esperimenti con encoder congelato e con Conditional Random Field (CRF) per la predizione della sequenza UPOS valutano rispettivamente il contributo del fine-tuning e della decodifica strutturata. L'estensione alla predizione delle caratteristiche morfologiche `FEATS` esplora inoltre l'apprendimento congiunto dei due compiti. Il protocollo sperimentale e le metriche adottate sono descritti nel Capitolo 2.

Capitolo 2

Metodologia

2.1 Definizione dell'esperimento

L'esperimento confronta le tre architetture schematizzate nella Figura 2.1 e descritte nella Tabella 2.1, con encoder congelato o addestrabile e classificazione UPOS lineare o strutturata. Ogni architettura viene valutata prima sul solo compito UPOS e poi sul compito congiunto UPOS e FEATS, ottenendo sei configurazioni che permettono di osservare la variazione delle prestazioni UPOS durante la prosecuzione del training.

Architettura	Encoder	UPOS	FEATS (fase 2)	Learning rate
BERT congelato	Pesi fissi	Lineare	Lineari	10^{-3}
BERT fine-tuned	Addestrabile	Lineare	Lineari	$2 \cdot 10^{-5}$
BERT + CRF	Addestrabile	Lineare e CRF	Lineari	$2 \cdot 10^{-5}$

Tabella 2.1: Architetture e learning rate. Le teste presenti e l'eventuale CRF sono sempre addestrabili.

Nella prima fase, a partire dal checkpoint BERT preaddestrato, l'ottimizzazione coinvolge la testa UPOS, l'eventuale CRF e, nelle architetture fine-tuned (i.e. con encoder addestrabile), l'encoder. Nella seconda fase, il checkpoint UPOS selezionato per la stessa architettura e lo stesso seed viene esteso aggiungendo le teste FEATS, inizializzate ex novo, mentre encoder, testa UPOS ed eventuale CRF mantengono i pesi appresi. L'addestramento prosegue su entrambi i compiti e, nelle architetture fine-tuned, le rispettive loss aggiornano l'encoder condiviso [14].

Ciascuna configurazione viene eseguita con i seed 42, 123 e 456, per un totale di 18 prove nel confronto finale, nove per fase, completando tutte le prove UPOS prima di avviare l'estensione a FEATS. L'impiego degli stessi seed permette di confrontare le configurazioni attraverso esecuzioni appaiate e di valutare la variabilità dei risultati mediante le statistiche descritte nella Sottosezione 2.7.3 [15].

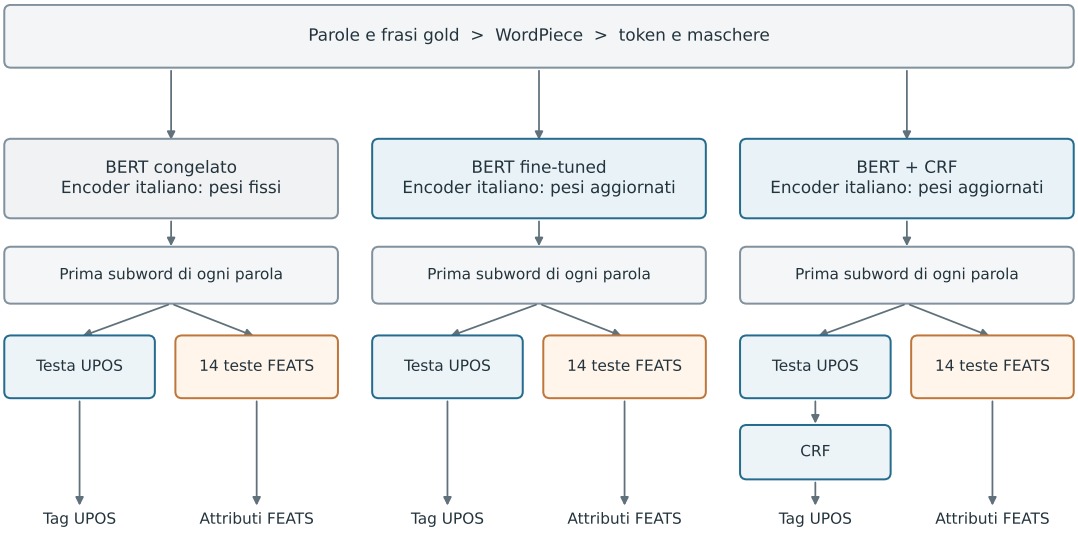


Figura 2.1: Schema delle tre architetture. Il colore grigio identifica l'encoder congelato. Le teste `FEATS` appartengono alla seconda fase.

2.2 Dati e annotazioni

2.2.1 Corpus e suddivisione

Si utilizza UD Italian-ISDT, release `r2.15`, scaricato dal repository ufficiale [5] e distribuito in file CoNLL-U (i.e. file testuali tabulari che riportano le parole delle frasi e le relative annotazioni linguistiche) [4]. Gli split ufficiali della Tabella 2.2 sono mantenuti, utilizzando il training per aggiornare i parametri e costruire gli inventari delle classi, il development set per selezionare il checkpoint e il test per la valutazione finale su dati esclusi dall'addestramento e dalla selezione.

Split	Frasi	Parole	Tag UPOS presenti
Train	13 121	276 014	17
Dev	564	11 907	17
Test	482	10 417	15

Tabella 2.2: Dimensioni degli split di UD Italian-ISDT 2.15; le parole corrispondono alle righe CoNLL-U con ID intero. Nel test sono assenti i tag `INTJ` e `PART`.

La lettura del corpus conserva le parole sintattiche identificate da un ID intero, escludendo le righe relative a token multiparola e nodi vuoti [4]. Il protocollo assume segmentazione gold (i.e. parole e confini di frase forniti dalle annotazioni del corpus) e confronta tutte le configurazioni sulle medesime parole, isolando la valutazione dell'etichettatura dalle operazioni di segmentazione. Un controllo delle sequenze di forme ha inoltre rilevato una frase identica tra training e sviluppo e nessuna tra training e test, mantenendo comunque la suddivisione ufficiale.

2.2.2 Rappresentazione di UPOS e FEATS

Gli inventari sono costruiti esclusivamente sul training e comprendono 17 tag UPOS e 14 attributi `FEATS`, per un totale di 40 coppie attributo-valore. Ogni parola riceve un'etichetta UPOS e un vettore morfologico con una componente per attributo, che assume uno dei valori osservati nel training oppure `<NONE>` quando l'attributo è assente.

Per esempio, `Gender=Masc|Number=Sing` viene rappresentato con `Masc` per Gender, `Sing` per Number e `<NONE>` per gli altri attributi; gli eventuali valori multipli sono ordinati e conservati come un'unica etichetta composta, secondo il formato CoNLL-U [4]. Le metriche mantengono anche le coppie gold assenti dal training, come `Typo=Yes` nel test, conteggiandole tra gli errori quando il modello non dispone della classe corrispondente.

2.3 Preprocessing e allineamento

Il tokenizer fast di `dbmdz/bert-base-italian-cased` viene applicato alle parole presegmentate, mantenendo la corrispondenza tra ciascuna subword e la parola di origine. Il vettore delle etichette gold (i.e. le annotazioni di riferimento fornite dal corpus) viene assegnato soltanto alla prima subword, escludendo token speciali, frammenti successivi e padding dalla loss e dalle metriche [16]; questa scelta mantiene una sola decisione supervisionata per parola, indipendentemente dal numero di frammenti prodotti dal tokenizer.

Nella variante CRF, gli score delle prime subword vengono raccolti in sequenze continue di parole e accompagnati da una maschera che identifica le posizioni valide, in modo che le transizioni siano apprese e decodificate tra parole consecutive. Il padding è dinamico per batch e tutte le frasi del corpus sono elaborate per intero; il preprocessing verifica che ogni parola sia coperta e interrompe l'esecuzione se tale condizione non è soddisfatta.

2.4 Modelli e funzione obiettivo

Le rappresentazioni dell'encoder alimentano una testa UPOS nella prima fase e 15 teste nella seconda, una per UPOS e 14 per `FEATS`, ciascuna con dropout di probabilità 0.1 seguito da una trasformazione lineare. Le teste condividono l'encoder ma hanno parametri distinti; quelle prive di CRF utilizzano cross-entropy mediata sulle parole valide durante l'addestramento e selezionano la classe con score massimo in predizione [16].

Nella variante CRF, gli score della testa UPOS costituiscono le emissioni di una catena lineare, combinate con parametri di transizione tra tag consecutivi per modellare le regolarità locali della sequenza. L'addestramento minimizza la log-verosimiglianza negativa della sequenza gold, normalizzata sul numero di parole del batch, mentre la decodifica Viterbi restituisce la sequenza con score complessivo massimo [17]. Il CRF opera soltanto su UPOS, lasciando lineari le teste `FEATS`.

Indicando con $\mathcal{A}$ l'insieme degli attributi morfologici, le funzioni obiettivo delle due fasi sono

$$\mathcal{L}^{(1)} = \mathcal{L}_{\text{UPOS}}, \qquad \mathcal{L}^{(2)} = \mathcal{L}_{\text{UPOS}} + \frac{\lambda}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} \mathcal{L}_a, \qquad \lambda = 1. \tag{2.1}$$

La media delle loss FEATS mantiene il peso complessivo del compito morfologico indipendente dal numero delle teste, mentre la presenza della loss UPOS nella seconda fase consente di proseguire l'apprendimento del compito principale durante l'introduzione delle nuove supervisioni. Il coefficiente λ rimane fisso per tutte le architetture e tutti i seed, includendo nell'apprendimento anche le decisioni di assenza rappresentate da `<NONE>`.

2.5 Procedura di addestramento

Il notebook di implementazione, che raccoglie preparazione dei dati, addestramento e valutazione, è disponibile nel repository del progetto [18]. L'ottimizzazione utilizza AdamW [19] attraverso il `Trainer` di Transformers [16], con i learning rate della Tabella 2.1 e le impostazioni comuni della Tabella 2.3. Il valore più elevato della variante congelata riguarda soltanto le teste, mentre nelle architetture fine-tuned il learning rate viene applicato anche all'encoder; il seed viene impostato prima della creazione del modello e controlla l'inizializzazione delle nuove teste, l'ordine dei batch e le operazioni stocastiche.

Parametro	Valore
Epoche per fase	10
Batch di training / valutazione	8 / 16 frasi
Weight decay	0.01
Warmup	10% dei passi
Clipping della norma dei gradienti	1.0

Tabella 2.3: Impostazioni comuni alle due fasi; dopo il warmup, il learning rate decresce linearmente.

La seconda fase avvia una nuova ottimizzazione AdamW e un nuovo scheduler per altre dieci epoche. Le variazioni UPOS vanno quindi interpretate considerando l'effetto congiunto dell'estensione multitask, della prosecuzione dell'addestramento e del criterio di selezione.

L'esecuzione su NVIDIA GeForce RTX 4070 SUPER, con 12 GB di memoria, utilizza precisione mista FP16 per accelerare il training e ridurre l'occupazione di memoria.

2.6 Selezione del checkpoint

Al termine di ogni epoca il modello viene valutato sul development set e, nella prima fase, viene selezionato il checkpoint che massimizza la macro-F1 UPOS sui 17 tag del

training. Nella seconda fase si utilizza invece il punteggio $S_{\text{dev}} = (F_{\text{U}} + F_{\text{F}})/2$, dove F_{U} indica la stessa macro-F1 UPOS e F_{F} la macro-F1 FEATS sulle coppie presenti nel development, escludendo `<NONE>`. La media assegna uguale peso ai due compiti nella selezione. Nella seconda fase viene salvato un checkpoint al termine di ciascuna delle dieci epoche; per la valutazione finale si utilizza quello con il punteggio migliore sul development set.

La Figura 2.2 mostra che la loss di training continua a diminuire, mentre quella sul development, dopo un calo iniziale, torna a crescere: prolungare l'addestramento non garantisce quindi una migliore generalizzazione. Nella prima fase i checkpoint selezionati per BERT fine-tuned corrispondono alle epoche 3, 4 e 5, rispettivamente per i seed 42, 123 e 456. Nella fase congiunta la macro-F1 `FEATS` cresce soprattutto nelle prime epoche e poi tende a stabilizzarsi, mentre quella UPOS oscilla; il punteggio medio dei due compiti seleziona le epoche 6, 5 e 10. La selezione sul development permette pertanto di utilizzare checkpoint diversi dall'ultimo, pur mantenendo un budget di dieci epoche per fase.

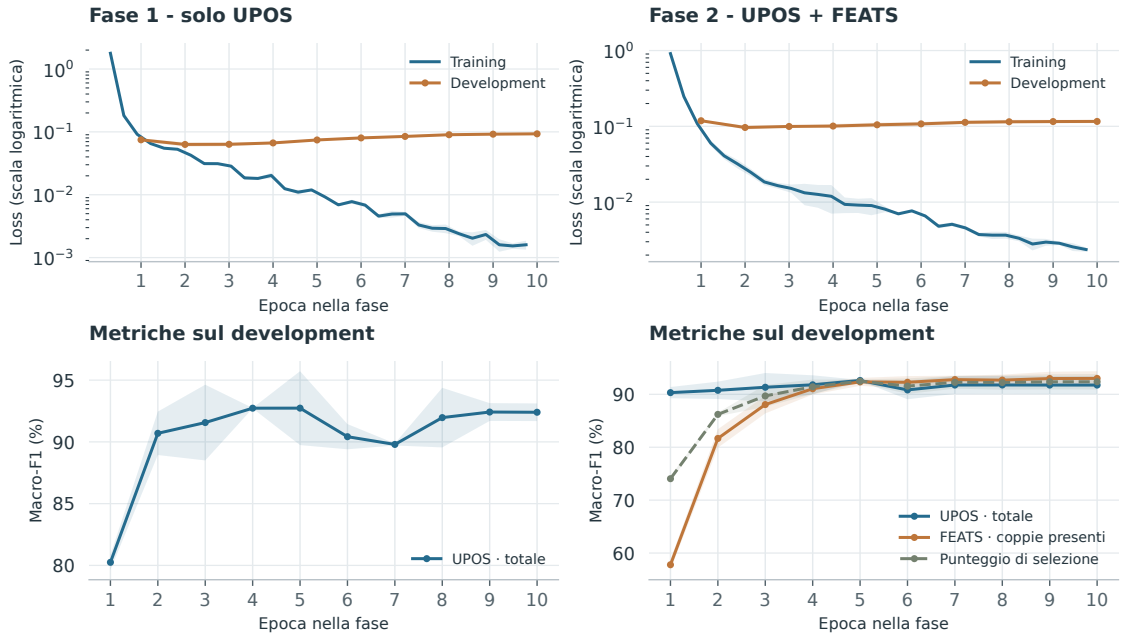


Figura 2.2: Andamento di BERT fine-tuned durante le due fasi: loss di training e sul development in alto, macro-F1 sul development in basso. Le linee indicano le medie sui tre seed e le aree ombreggiate una deviazione standard campionaria.

2.7 Valutazione

Si valutano UPOS per tutte le configurazioni e, nella seconda fase, anche FEATS e accuracy congiunta. Le stesse metriche sono applicate a UDPipe 2, verificando ordine ed etichette gold.

2.7.1 Metriche UPOS

L'accuracy misura la frazione di parole con tag corretto e descrive la prestazione complessiva, alla quale viene affiancata la macro-F1 per attribuire lo stesso peso alle categorie, comprese quelle meno frequenti. Per ciascun tag k si conteggiano veri positivi (TP_k), falsi positivi (FP_k) e falsi negativi (FN_k), dai quali si ricava l'F1 come media armonica di precision e recall [20]:

$$F_{1,k} = \frac{2TP_k}{2TP_k + FP_k + FN_k}, \qquad F_1^{\text{macro}} = \frac{1}{|C|} \sum_{k \in C} F_{1,k}. \tag{2.2}$$

La versione *totale* include i 17 tag del training, mentre quella sulle classi *presenti* considera soltanto i tag con supporto gold positivo nello split valutato, che nel test sono 15 per l'assenza di `INTJ` e `PART`. Ponendo a zero gli F1 con denominatore nullo, la macro-F1 totale sul test ha quindi un massimo teorico di $15/17 \simeq 88{,}24\%$ anche con tutte le predizioni corrette, mentre quella sulle classi presenti mantiene come massimo il 100%.

Con parole allineate e un solo tag gold e predetto per parola, precision, recall e F1 con media *micro* coincidono con l'accuracy [20]. Infatti, sommando i conteggi su tutti i tag, ogni errore produce un falso positivo e un falso negativo: precision e recall risultano entrambe pari alla frazione di parole corrette, e lo stesso vale per la loro media armonica. Questa equivalenza non vale per la macro-F1, che media i punteggi delle singole categorie. Il riepilogo utilizza quindi accuracy e macro-F1; precision e recall per classe restano disponibili per l'analisi degli errori.

2.7.2 Metriche FEATS

L'accuracy FEATS principale richiede che l'intero insieme di coppie attributo-valore di una parola coincida con quello gold, valutando congiuntamente la presenza degli attributi e la correttezza dei loro valori. Indicando con G_i e P_i gli insiemi gold e predetto per la parola i, si calcola

$$\text{Acc}_{\text{FEATS}} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[G_i = P_i], \tag{2.3}$$

includendo anche gli insiemi vuoti, mentre una misura aggiuntiva conta le decisioni corrette sulle coppie parola-attributo e comprende la classe `<NONE>`. Quest'ultima misura viene interpretata insieme all'accuracy dell'insieme completo, poiché la frequenza degli attributi assenti può incidere sensibilmente sul suo valore; gli attributi valutati sono l'unione di quelli di training, gold e predetti.

La macro-F1 tratta ogni coppia attributo-valore (e.g. `Gender=Fem`) come una classe e ne conteggia veri positivi, falsi positivi e falsi negativi sulla presenza nelle singole parole, escludendo `<NONE>` dalla media. La versione *totale* comprende le 40 coppie del training e le ulteriori coppie osservate nelle annotazioni gold o nelle predizioni, mentre quella sulle classi *presenti* considera soltanto le coppie con supporto gold positivo, mantenendo nel conteggio degli errori anche le annotazioni sconosciute al modello.

Si riporta inoltre l'accuracy congiunta, definita come la frazione di parole per le quali risultano corretti sia UPOS sia l'intero insieme FEATS, affiancando alle misure complessive le metriche dei singoli attributi per esaminarne le differenze di comportamento.

2.7.3 Aggregazione e analisi degli errori

Per ogni configurazione si riportano media e deviazione standard campionaria sui tre seed [15], conservando le misure individuali. Le variazioni UPOS tra le fasi sono appaiate per architettura e seed ed espresse in punti percentuali. L'analisi degli errori considera confusioni UPOS, falsi positivi e negativi delle coppie `FEATS` e metriche per classe.

2.8 Confronto con UDPipe 2

UDPipe 2 `italian-isdt-ud-2.15-241121` riceve le stesse parole e gli stessi confini di frase gold di BERT, eliminando dall'input le annotazioni grammaticali [13]. Le predizioni vengono allineate e valutate con le metriche definite nella Sezione 2.7. Per `FEATS` si considera l'intero insieme degli attributi.

L'incertezza degli scarti UPOS tra BERT e UDPipe viene stimata mediante bootstrap appaiato per frase (i.e. estrazione casuale delle frasi con reinserimento, usando gli stessi campioni per entrambi i sistemi) [21]. Si generano 10 000 campioni delle 482 frasi del test, mantenendo fisso l'inventario delle 15 classi presenti. Dagli scarti ottenuti si ricava l'intervallo centrale del 95%, separatamente per fase e seed. Gli intervalli assumono indipendenza tra frasi e descrivono l'incertezza sul test per modelli già addestrati; non includono la variabilità del training.

2.9 Riproducibilità

Per ogni prova si conservano il modello selezionato, la cronologia di addestramento, le predizioni e le metriche. Si registrano inoltre revisioni, inventari delle etichette, seed, iperparametri, versioni software e origine dei pesi UPOS. Gli hash dei dati, ossia impronte digitali dei file, permettono di verificare che gli esperimenti utilizzino gli stessi dati. Codice e istruzioni sono disponibili nel repository del progetto [18].

Capitolo 3

Risultati

La valutazione del modello fine-tuned per UPOS è incentrata sul confronto con UDPipe 2, riprodotto nelle stesse condizioni sul benchmark UD Italian-ISDT 2.15, e sul posizionamento rispetto ai risultati pubblicati. Il test comprende 10 417 parole (Tabella 2.2); per le configurazioni BERT si riportano media e deviazione standard campionaria sui tre seed delle 18 prove finali. I confronti tra architetture e l'estensione `FEATS` completano l'analisi del sistema.

3.1 Confronto UPOS con UDPipe 2

BERT fine-tuned sul solo compito UPOS raggiunge un'accuracy del $98{,}74 \pm 0{,}11\%$, rispetto al 98,75% di UDPipe 2. Dopo l'estensione congiunta a UPOS e `FEATS`, la stessa architettura raggiunge il $98{,}86 \pm 0{,}09\%$. La Figura 3.1 evidenzia il confronto sulle due metriche principali, secondo il protocollo della Sezione 2.8.

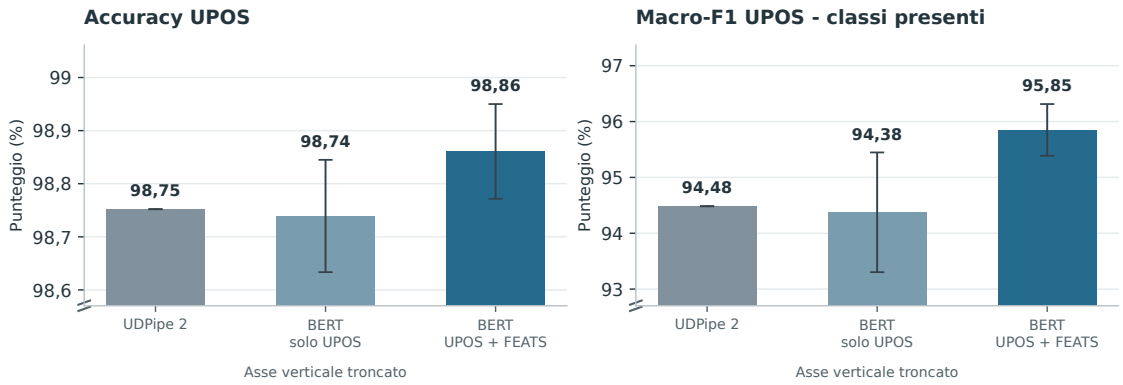


Figura 3.1: Accuracy e macro-F1 UPOS sulle classi presenti per UDPipe 2 e BERT fine-tuned, prima e dopo la fase congiunta con `FEATS`. Per BERT le colonne mostrano la media sui tre seed e le barre di errore la deviazione standard; UDPipe corrisponde a una sola esecuzione. Gli assi verticali sono troncati per evidenziare gli scarti.

La Tabella 3.1 riporta i risultati delle tre architetture in entrambe le fasi.

Architettura	Accuracy	Macro-F1 presenti	Macro-F1 totale
UDPipe 2 (UD 2.15)	98,75	94,48	83,37
Fase 1: UPOS			
BERT congelato	$96{,}53 \pm 0{,}21$	$90{,}23 \pm 0{,}30$	$79{,}62 \pm 0{,}27$
BERT fine-tuned	$98{,}74 \pm 0{,}11$	$94{,}38 \pm 1{,}07$	$83{,}27 \pm 0{,}95$
BERT + CRF	$98{,}77 \pm 0{,}09$	$94{,}86 \pm 1{,}31$	$83{,}70 \pm 1{,}16$
Fase 2: UPOS + `FEATS`			
BERT congelato	$96{,}76 \pm 0{,}03$	$90{,}77 \pm 0{,}37$	$80{,}09 \pm 0{,}32$
BERT fine-tuned	$98{,}86 \pm 0{,}09$	$95{,}85 \pm 0{,}46$	$84{,}57 \pm 0{,}41$
BERT + CRF	$98{,}79 \pm 0{,}07$	$95{,}66 \pm 0{,}50$	$84{,}41 \pm 0{,}44$

Tabella 3.1: Risultati UPOS sul medesimo test UD Italian-ISDT 2.15 con segmentazione gold, espressi in percentuale come media ± deviazione standard. Per UDPipe si riporta un'unica esecuzione; la macro-F1 sulle classi presenti utilizza 15 tag, quella totale tutti i 17 tag.

Lo scarto medio di accuracy rispetto a UDPipe è di circa $-0{,}01$ punti percentuali nella prima fase e $+0{,}11$ nella seconda. UDPipe commette 130 errori, mentre BERT fine-tuned della seconda fase ne commette 123, 125 e 108 nei tre seed. La differenza corrisponde quindi a poche parole su oltre diecimila e viene esaminata anche attraverso gli intervalli del confronto appaiato.

La macro-F1 sulle 15 classi presenti rende più articolato il confronto: nella prima fase BERT fine-tuned ottiene il 94,38%, contro il 94,48% di UDPipe, con accuracy medie quasi coincidenti. Dopo la fase congiunta raggiunge il 95,85%, con uno scarto medio di 1,37 punti rispetto al riferimento. Circa 1,16 punti di questo scarto provengono dalla categoria `X`, presente soltanto 13 volte, il cui F1 passa dal 37,50% di UDPipe al 54,92% medio di BERT. Poiché la macro-F1 assegna lo stesso peso a ciascuna classe, la migliore prestazione di BERT su `X` incide molto sul punteggio complessivo. Questo non indica che BERT riconosca meglio tutte le classi rare o che il miglioramento sia uniforme tra i tag. La macro-F1 sui 17 tag segue lo stesso andamento, con la differenza di scala dovuta alle due classi assenti dal test descritta nella Sottosezione 2.7.1.

La Tabella 3.2 quantifica l'incertezza degli scarti di BERT fine-tuned rispetto a UDPipe, usando il ricampionamento per frase descritto nella Sezione 2.8. Gli intervalli, più ampi per la macro-F1 anche per il peso delle categorie con pochi esempi, non permettono di stabilire una superiorità complessiva di BERT. Le prestazioni osservate sono comunque nel complesso in linea con quelle di UDPipe.

	Accuracy		Macro-F1 (15 classi)	
Seed	Scarto	Intervallo al 95%	Scarto	Intervallo al 95%
Fase 1: UPOS				
42	−0,13	[−0,36; +0,07]	−0,79	[−3,08; +1,17]
123	+0,06	[−0,16; +0,28]	−0,66	[−4,08; +1,70]
456	+0,04	[−0,16; +0,23]	+1,13	[−1,10; +3,76]
Fase 2: UPOS + `FEATS`				
42	+0,07	[−0,12; +0,25]	+0,89	[−1,54; +4,03]
123	+0,05	[−0,16; +0,25]	+1,40	[−1,54; +4,40]
456	+0,21	[+0,04; +0,39]	+1,81	[−0,17; +4,75]

Tabella 3.2: Scarti UPOS di BERT fine-tuned rispetto a UDPipe, in punti percentuali, con intervalli bootstrap appaiati al 95%. Le stime sono separate per seed e basate su 10 000 ricampionamenti di frasi; gli intervalli non sono corretti per confronti multipli.

3.2 Posizionamento rispetto alla letteratura

Il confronto con la letteratura considera sistemi pertinenti al PoS tagging italiano, con particolare riferimento ai risultati su Italian-ISDT. La Tabella 3.3 raccoglie sia encoder Transformer italiani sia modelli linguistici generativi di grandi dimensioni (LLM), riportando per ciascuno la metrica e le condizioni di valutazione disponibili.

Sistema e fonte	UPOS (%)	Metrica e condizioni dichiarate
UmBERTo [22]	98,78 ± 0,08	Accuracy; UD-ISDT 2.5; almeno 5 esecuzioni.
BERT italiano *cased* [23]	98,84 ± 0,06	Accuracy; ISDT, commit `f20fa2b`; 5 esecuzioni.
BERT italiano XXL *cased* [23]	98,90 ± 0,03	Accuracy; stesso protocollo di BERT italiano.
UmBERTo Commoncrawl *cased* [24]	98,977	Accuracy; Italian-ISDT, release non specificata.
Gemma-2-9B multilingue [25]	98,8	F1 UPOS; UD 2.15; output generati.
LLM con LoRA, quattro modelli [26]	98,3–98,8	F1 UPOS; UD 2.15; LoRA, generazione in due passi.

Tabella 3.3: Riferimenti di letteratura per UPOS su Italian-ISDT. I valori sono quelli delle fonti e non costituiscono una graduatoria su un protocollo comune; i risultati dei repository degli autori sono distinti da quelli degli articoli.

Le accuracy del modello sviluppato sono vicine ai risultati dei Transformer italiani riportati nella Tabella 3.3.

Le differenze architetturali riguardano anche il modo in cui viene utilizzata l'informazione morfologica. Nel sistema di Tamburini [27], valutato sui benchmark EVALITA, una rete BiLSTM-CRF riceve in ingresso rappresentazioni lessicali ar-

ricchite con le analisi prodotte da AnIta, un analizzatore morfologico per l'italiano. Nel presente progetto, invece, la morfologia viene appresa come compito supervisionato aggiuntivo mediante le teste `FEATS`, senza fornire al modello le analisi di uno strumento esterno.

Le condizioni sui dati sono riportate per contestualizzare i valori della tabella. Il confronto dei file ISDT del commit `f20fa2b`, usato da Schweter, con UD 2.15 conferma che il test conserva le stesse 482 frasi e 10 417 parole, con forme e confini di frase identici, ma contiene cinque parole con etichetta UPOS diversa; anche training e development presentano differenze, come verificabile con lo script di confronto incluso nel repository del progetto [18].

I lavori di Matsuda et al. [25] e Matsuda e Asahara [26] ampliano il confronto ai modelli generativi su UD 2.15. Nel secondo, gli LLM sono adattati mediante Low-Rank Adaptation (LoRA), una tecnica che limita i parametri aggiornati, e generano prima i tag UPOS e poi le dipendenze sintattiche tra le parole, senza ripetere le forme nell'output.

3.3 Confronti interni tra le configurazioni BERT

I confronti interni mostrano il contributo del fine-tuning: nella prima fase l'accuracy passa dal 96,53% dell'encoder congelato al 98,74% di BERT fine-tuned. L'aggiunta del CRF mantiene un'accuracy simile e migliora la macro-F1 sulle classi presenti, con un contributo della categoria rara `X`, discusso nella Sezione 3.5. Nelle prove svolte, la variante con CRF richiede più tempo di BERT fine-tuned senza CRF in entrambe le fasi. L'addestramento, comprensivo di valutazioni e salvataggi, resta nell'ordine di decine di minuti.

La Figura 3.2 riassume il passaggio alla seconda fase: l'incremento medio di accuracy rimane contenuto, mentre la macro-F1 sulle classi presenti mostra un miglioramento più evidente, soprattutto per BERT fine-tuned.

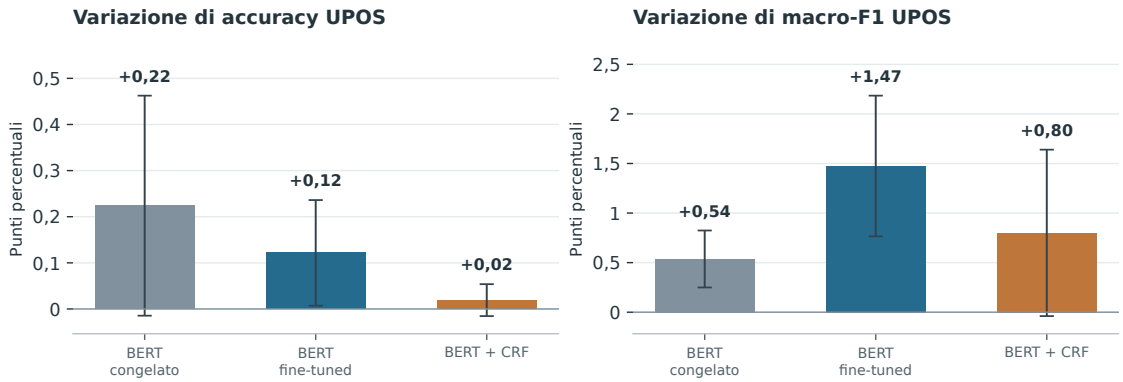


Figura 3.2: Variazioni UPOS tra la fase congiunta e quella sul solo UPOS, calcolate appaiando i risultati dello stesso seed. Le colonne indicano la variazione media e le barre di errore la deviazione standard campionaria sui tre seed. La macro-F1 considera le classi presenti; le scale dei due pannelli sono differenti.

3.4 Estensione alla predizione FEATS

La Figura 3.3 mette a confronto BERT fine-tuned e UDPipe 2 sulle due metriche morfologiche principali, definite nella Sottosezione 2.7.2; la Tabella 3.4 completa il quadro con tutte le configurazioni della seconda fase.

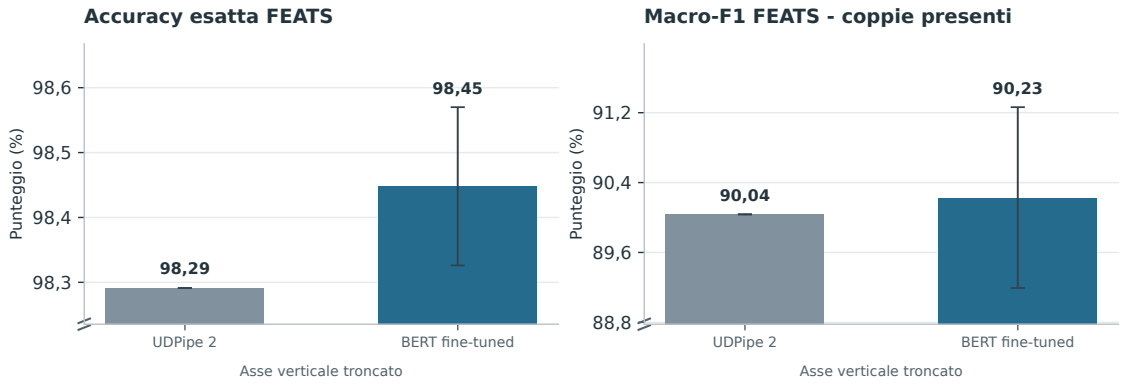


Figura 3.3: Accuracy esatta e macro-F1 sulle coppie FEATS presenti: confronto tra BERT fine-tuned e UDPipe 2. Per BERT si riportano media e deviazione standard sui tre seed; per UDPipe una sola esecuzione. Gli assi verticali sono troncati.

Sistema	Accuracy FEATS	Macro-F1 presenti	Accuracy congiunta	Macro-F1 totale
UDPipe 2	98,29	90,04	97,94	83,45
BERT congelato	$87{,}98 \pm 0{,}12$	$83{,}67 \pm 0{,}34$	$86{,}86 \pm 0{,}11$	$77{,}54 \pm 0{,}31$
BERT fine-tuned	$98{,}45 \pm 0{,}12$	$90{,}23 \pm 1{,}04$	$98{,}05 \pm 0{,}17$	$83{,}63 \pm 0{,}96$
BERT + CRF	$98{,}46 \pm 0{,}18$	$90{,}17 \pm 1{,}41$	$98{,}02 \pm 0{,}16$	$83{,}57 \pm 1{,}30$

Tabella 3.4: Risultati morfologici sul test UD Italian-ISDT 2.15 con segmentazione gold. Per BERT si riportano media ± deviazione standard sui tre seed; per UDPipe un'unica esecuzione. Tutti i valori sono percentuali e ricalcolati con le stesse convenzioni. Le macro-F1 considerano 41 coppie attributo-valore complessive e 38 presenti nel test.

Come mostra la Figura 3.3, BERT fine-tuned raggiunge valori medi lievemente superiori a UDPipe sia per l'accuracy esatta FEATS sia per la macro-F1 sulle coppie presenti, con scarti rispettivamente di circa 0,16 e 0,19 punti percentuali. La macro-F1 presenta però una deviazione standard di 1,04 punti sui tre seed, maggiore dello scarto medio rispetto al riferimento; non emerge quindi un vantaggio uniforme nelle singole esecuzioni.

Nei confronti interni, l'accuracy FEATS passa dall'87,98% del modello congelato al 98,45% di BERT fine-tuned e la macro-F1 sulle coppie presenti dall'83,67% al 90,23%. Gli scarti tra i due modelli fine-tuned rimangono contenuti: il CRF presenta un'accuracy FEATS media maggiore di circa 0,01 punti e risulta lievemente migliore in due seed su tre, mentre la macro-F1 media favorisce di circa 0,06 punti BERT

senza CRF. Poiché il CRF non modifica direttamente le teste morfologiche, queste differenze riflettono l'apprendimento congiunto e la selezione del checkpoint.

L'accuracy sulle singole decisioni parola-attributo raggiunge il 99,85%. Per ogni parola si valuta ciascun attributo, contando come corretta anche la previsione `<NONE>` quando l'attributo è assente. Poiché molte decisioni sono di questo tipo, il loro peso sul totale può mantenere alta l'accuracy anche se alcuni valori morfologici presenti vengono predetti in modo errato. Anche l'accuracy esatta include le 4 055 parole con insieme gold vuoto, considerate corrette quando il modello non predice alcun attributo. La macro-F1 sulle coppie presenti, che esclude `<NONE>`, aiuta quindi a valutare le caratteristiche effettivamente annotate.

3.5 Analisi degli errori

Le principali confusioni UPOS coinvolgono nomi comuni, nomi propri e aggettivi: sommando le tre esecuzioni di BERT fine-tuned della prima fase, si osservano 52 assegnazioni da `NOUN` a `PROPN`, 36 nella direzione opposta e 33 da `NOUN` a `ADJ`. La Figura 3.4 mostra che questi scambi restano prevalenti dopo l'estensione morfologica.

La categoria `X`, presente soltanto 13 volte, chiarisce la sensibilità della macro-F1: nella prima fase il suo F1 medio passa dal 37,63% con classificazione lineare al 41,67% con CRF, contribuendo per circa 0,27 punti al vantaggio macro complessivo di 0,48 punti della seconda architettura. Dopo la fase congiunta, i valori raggiungono rispettivamente il 54,92% e il 53,87%, contribuendo agli incrementi della media per categoria a fronte di variazioni ridotte dell'accuracy.

Per `FEATS`, gli errori in valore assoluto si concentrano soprattutto su numero e genere, che hanno elevato supporto: e.g., `Number=Sing` accumula 105 falsi positivi e 101 falsi negativi nelle tre prove di BERT fine-tuned, pur mantenendo un F1 medio del 99,16%. Restano più difficili alcune coppie rare, tra cui `Foreign=Yes`, con 11 occorrenze e F1 medio del 38,10%, e `Degree=Abs`, con tre occorrenze e F1 medio del 38,89%. Quest'ultima presenta valori del 50,00%, 0,00% e 66,67% nei tre seed, mostrando come poche decisioni possano incidere sulla macro-F1. `PronType=Exc`, presente una sola volta, riceve F1 nullo in tutte le configurazioni BERT; anche l'unica occorrenza di `Typo=Yes` resta errata, poiché la coppia è assente dall'inventario di training.

UDPipe riconosce due delle tre occorrenze di `Degree=Abs`, con un F1 del 57,14%. Su questa coppia supera BERT fine-tuned di 18,25 punti di F1, equivalenti a circa 0,48 punti di macro-F1 sulle 38 coppie presenti. Le differenze sulle altre coppie compensano questo contributo e portano BERT a una macro-F1 media complessiva lievemente maggiore del riferimento.

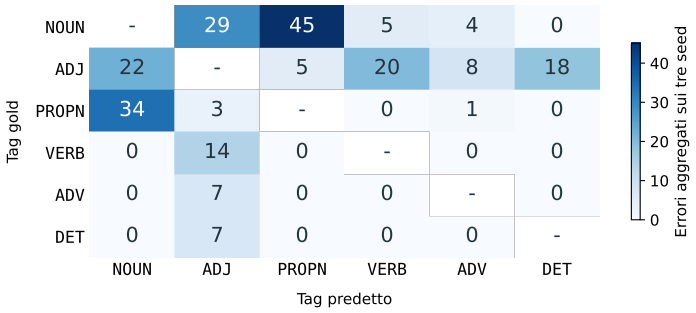


Figura 3.4: Confusioni UPOS di BERT fine-tuned: righe gold, colonne predette. Ogni cella conta gli errori aggregati sui tre seed; il blu più scuro indica più errori, ad esempio 45 da `NOUN` a `PROPN`. La diagonale delle predizioni corrette è esclusa.

3.5.1 Esempi contestuali

La Tabella 3.5 raccoglie quattro casi illustrativi, riportando le predizioni di BERT fine-tuned della fase congiunta per il seed 123. Comprendono successi ed errori di entrambi i sistemi e un'omissione morfologica, senza costituire un campione rappresentativo. Le predizioni non sono sempre identiche fra seed: con il seed 42 BERT riconosce correttamente *Ministro* come `PROPN` e *conclusa* come `ADJ`; inoltre, i seed 42 e 456 predicono correttamente `Degree=Abs` per *Recentissima*.

Estratto	Campo	Gold	UDPipe	BERT (123)
Il Presidente **eletto** nomina e revoca i componenti della Giunta.	UPOS	`ADJ`	`VERB`	`ADJ`
Chi è il **Ministro** degli Esteri italiano?	UPOS	`PROPN`	`PROPN`	`NOUN`
... l'operazione è considerata già **conclusa**.	UPOS	`ADJ`	`VERB`	`VERB`
Recentissima anche la creazione di "Splatter" ...	`Degree`	`Abs`	`Abs`	`<NONE>`

Tabella 3.5: Esempi sul test: parola bersaglio in grassetto ed estratti abbreviati con puntini. L'ultima riga confronta `Degree`; `<NONE>` ne indica l'assenza.

Nel gold, *eletto* identifica *Presidente* come aggettivo: BERT riproduce questa annotazione, mentre UDPipe assegna `VERB` e caratteristiche di participio. Anche *conclusa* è annotata come `ADJ`, ma entrambi i sistemi la trattano come verbo; i due casi mostrano come la stessa origine participiale possa accompagnarsi a decisioni diverse nel contesto.

Per *Ministro*, UDPipe segue il gold `PROPN`, mentre BERT assegna `NOUN`: il caso richiama il trattamento dei titoli istituzionali nel treebank, rispetto al quale viene calcolata la correttezza. In *Recentissima*, invece, entrambi riconoscono `ADJ`, ma BERT omette `Degree=Abs` pur predicendo correttamente genere e numero; distinguere la categoria grammaticale dall'insieme morfologico permette così di localizzare l'errore.

Capitolo 4

Conclusioni

Il progetto ha adattato BERT italiano al PoS tagging sul benchmark UD Italian-ISDT 2.15, utilizzando gli split ufficiali e la segmentazione gold. Con dieci epoche di addestramento per fase e selezione sul development, il modello fine-tuned raggiunge un'accuracy UPOS di circa il 98,74% nella prima fase e, dopo l'estensione congiunta a UPOS e `FEATS`, ottiene un'accuracy UPOS di circa il 98,86% e una macro-F1 sulle classi presenti di circa il 95,85%, rispetto al 98,75% e al 94,48% di UDPipe 2, evidenziando prestazioni sostanzialmente allineate fra i due sistemi.

Per la morfologia, BERT fine-tuned raggiunge il 98,45% di accuracy esatta `FEATS` e il 90,23% di macro-F1 sulle coppie presenti, rispetto al 98,29% e al 90,04% di UDPipe. Gli scarti medi sono contenuti e, soprattutto per la macro-F1, si accompagnano a una variabilità tra seed maggiore del vantaggio medio. Alla buona correttezza complessiva si affiancano difficoltà sulle caratteristiche rare, che incidono maggiormente sulla misura per categoria.

I confronti interni mostrano che il fine-tuning migliora nettamente le prestazioni rispetto all'encoder congelato, mentre il CRF offre un beneficio più circoscritto sulla macro-F1 nella prima fase, legato in parte alla categoria rara `X`, senza un miglioramento uniforme tra compiti e seed. L'estensione alla morfologia si accompagna a un miglioramento delle prestazioni UPOS, in particolare della macro-F1 sulle classi presenti.

Un possibile sviluppo consiste nell'aggiungere un CRF a ciascuna testa `FEATS`, estendendo la decodifica strutturata alle sequenze dei singoli attributi; i benefici contenuti osservati su UPOS suggeriscono che anche questa estensione potrebbe produrre guadagni limitati. Miglioramenti più consistenti potrebbero richiedere un'architettura capace di modellare esplicitamente le dipendenze tra categorie grammaticali e attributi morfologici, rappresentando anche i vincoli di compatibilità tra le etichette, oltre alle relazioni tra parole consecutive.

La valutazione su un solo corpus annotato, con segmentazione gold e tre seed, circoscrive le conclusioni, suggerendo ulteriori analisi statistiche appaiate e valutazioni su altri corpora o da testo grezzo.

Bibliografia

[1] Christopher D. Manning. Part-of-speech tagging from 97% to 100%: Is it time for some linguistics? In *Computational Linguistics and Intelligent Text Processing*, volume 6608 of *Lecture Notes in Computer Science*, pages 171–189. Springer, 2011. doi:10.1007/978-3-642-19400-9_14.

[2] Universal Dependencies contributors. Universal dependencies: Framework and documentation. Sito ufficiale del progetto Universal Dependencies. URL: https://universaldependencies.org/.

[3] Universal Dependencies contributors. Universal POS tags. Universal Dependencies v2 – Linee guida per l'annotazione delle categorie grammaticali. URL: https://universaldependencies.org/u/pos/.

[4] Universal Dependencies contributors. CoNLL-U format. Specifiche di Universal Dependencies v2. URL: https://universaldependencies.org/format.html.

[5] Universal Dependencies contributors. UD Italian-ISDT. Treebank, release r2.15, 2024. URL: https://github.com/UniversalDependencies/UD_Italian-ISDT/tree/r2.15.

[6] Haibo He and Edwardo A. Garcia. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284, 2009. doi:10.1109/TKDE.2008.239.

[7] Barbara Plank, Anders Søgaard, and Yoav Goldberg. Multilingual part-of-speech tagging with bidirectional long short-term memory models and auxiliary loss. In *Proceedings of ACL 2016 (Volume 2: Short Papers)*, pages 412–418. Association for Computational Linguistics, 2016. doi:10.18653/v1/P16-2067.

[8] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT 2019*, pages 4171–4186. Association for Computational Linguistics, 2019. doi:10.18653/v1/N19-1423.

[9] Yoav Goldberg. A primer on neural network models for natural language processing. *Journal of Artificial Intelligence Research*, 57:345–420, 2016. doi:10.1613/jair.4992.

[10] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeff Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems*, volume 26. Curran Associates, Inc., 2013. URL: https://papers.nips.cc/paper_files/paper/2013/hash/9aa42b31882ec039965f3c4923ce901b-Abstract.html.

[11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL: https://papers.nips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

[12] DBMDZ. BERT base italian cased. Model card, Hugging Face. Consultato il 7 novembre 2026. URL: https://huggingface.co/dbmdz/bert-base-italian-cased.

[13] ÚFAL, Charles University. UDPipe 2 models: Universal Dependencies 2.15. Documentazione dei modelli. Consultato il 7 novembre 2026. URL: https://ufal.mff.cuni.cz/udpipe/2/models.

[14] Dan Kondratyuk and Milan Straka. 75 languages, 1 model: Parsing Universal Dependencies universally. In *Proceedings of EMNLP-IJCNLP 2019*, pages 2779–2795. Association for Computational Linguistics, 2019. doi:10.18653/v1/D19-1279.

[15] Nils Reimers and Iryna Gurevych. Reporting score distributions makes a difference: Performance study of LSTM-networks for sequence tagging. In *Proceedings of EMNLP 2017*, pages 338–348. Association for Computational Linguistics, 2017. doi:10.18653/v1/D17-1035.

[16] Hugging Face. Token classification. Documentazione di Transformers, versione 4.46.3. URL: https://huggingface.co/docs/transformers/v4.46.3/tasks/token_classification.

[17] John Lafferty, Andrew McCallum, and Fernando C. N. Pereira. Conditional random fields: Probabilistic models for segmenting and labeling sequence data. In *Proceedings of ICML 2001*, pages 282–289. Morgan Kaufmann, 2001. URL: https://repository.upenn.edu/cis_papers/159.

[18] Pierre Errico. Pos tagging italiano con bert: codice e materiali del progetto. Repository GitHub, 2026. URL: https://github.com/pierreerrico/bert-italian-pos-tagging.

[19] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL: https://arxiv.org/abs/1711.05101.

[20] scikit-learn developers. Metrics and scoring: Quantifying the quality of predictions. Documentazione di scikit-learn, versione 1.8. Consultato l'8 novembre 2026. URL: https://scikit-learn.org/1.8/modules/model_evaluation.html.

[21] Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1383–1392, 2018. URL: https://aclanthology.org/P18-1128/, doi:10.18653/v1/P18-1128.

[22] Fabio Tamburini. How BERTology changed the state-of-the-art also for italian NLP. In *Proceedings of CLiC-it 2020*, pages 313–319. CEUR Workshop Proceedings, 2020. URL: https://aclanthology.org/2020.clicit-1.48/.

[23] Stefan Schweter. Italian BERT and ELECTRA models. Repository degli autori, 2020. Test ISDT al commit f20fa2b. URL: https://github.com/stefan-it/italian-bertelectra.

[24] Loreto Parisi, Simone Francia, and Paolo Magnani. UmBERTo: an italian language model trained with whole word masking. Repository degli autori, 2020. Consultato il 9 novembre 2026. URL: https://github.com/musixmatchresearch/umberto.

[25] Hiroshi Matsuda, Chunpeng Ma, and Masayuki Asahara. Step-by-step instructions and a simple tabular output format improve the dependency parsing accuracy of LLMs. In *Proceedings of IWPT (SyntaxFest 2025)*, pages 11–19. Association for Computational Linguistics, 2025. URL: https://aclanthology.org/2025.iwpt-1.2/.

[26] Hiroshi Matsuda and Masayuki Asahara. Probing the dynamics of syntactic ability acquisition throughout LLM pretraining. In *Proceedings of the Ninth Workshop on Universal Dependencies (UDW 2026)*, pages 1–48. ELRA Language Resources Association (ELRA), 2026. URL: https://aclanthology.org/2026.udw-1.1/, doi:10.63317/2n8gkp49p3wg.

[27] Fabio Tamburini. (better than) state-of-the-art PoS-tagging for italian texts. In *Proceedings of the Third Italian Conference on Computational Linguistics (CLiC-it 2016)*, pages 294–298, 2016. URL: https://aclanthology.org/2016.clicit-1.55/.